CS 370 Computer Imaging

Programming Assignment - 3
Implementing the Fourier Transform (Due : Week 14, Sunday, 23:59)

Objective : In this project, you will implement the Fourier transform (forward and backward) using the direct method and the Fast-Fourier Transform (decimation-in-time method).

Part (A) Direct Implementation (10 points)
In this part of the assignment, implement the code for calculating a 2D Fourier Transform using the following methods:

(a) Direct method - two nested for-loops (4 points)

Implement the DFT **as per the definition of the formula**. You will implement two for-loops that march along the rows and columns of the input image FOR EVERY POINT IN THE OUTPUT. That means, you will march through the entire image for every pair of values (u,v) in the output range. This effectively makes your solution four nested for-loops (which is considerably worse than the part (b) below.)

(b) Separable method - As a concatenation of two 1D Fourier Transforms. (6 points)

You will **first perform the 1D Fourier Transform on the rows of the image**, store the intermediate result, and then perform **the column-wise Fourier transform on the intermediate values** to complete the calculation of the 2D transform.

Part (B) Fast Fourier Transform (20 points)
In this part of the assignment, you will implement the time-decimation Fast Fourier Transform as discussed in class. Some helpful hints to develop the functionality:

(a) You will need to pre-process the input data in order to align the data points according to the Butterfly configuration. Use the **bit-reversal algorithm** to do so.

(b) Your output should be an in-place computation (to save memory).

(c) You will need to post-process the output of the FFT (Complex Numbers) into a scalar representation in order to display it as an image. You may use the “log” transform to achieve this and then display the Fourier Spectrum.

For both the parts (A) and (B), you should display the following :

(a) The original image - displayed normally in the application window.
(b) The Fourier Spectrum - displayed in a separate window/panel.

(c) The Reconstructed image - this image is calculated by applying the inverse Fourier transform to the Fourier spectrum. Ideally, this should be the same image as the input image. Display this image under a separate window/panel as well.

Some implementation points to consider:
You will save considerable coding effort if you could reuse your forward Fourier Transform code to calculate the inverse transform as well. **How will you implement the Inverse Fourier transform using the Forward Fourier Transform?**
Your Fourier Spectrum should be centered in the location (M/2, N/2) and not at (0,0). **How will you shift the Fourier Spectrum F(u,v) by manipulating the input signal f(x,y)?**

Part (C) Displaying the results of the FFT (10 points)

Use intensity / color slicing instead of a "log" transform to display the Fourier Spectrum in pseudo-color representation. In this case, you will represent a range of values in the Fourier spectrum by a single color - say [0, 10.0) is red, [10.0 to 1000.0) is dark blue etc. The user should be able to input the number of "bins" for slicing. Provide examples in the README document. (10 points)

Part (D) Analysis of the implementation (20 points)

Write up a short description of your solution to the project. Describe the classes / methods you have implemented. Compare and contrast the runtimes of all three algorithms on at least 5 images from the dataset - use average value of the runtimes for display. Use Excel or matplotlib charts to visualize the timing comparisons. You should use appropriate scale in order to capture the variation in the timings. The comparison method is described below.

Comparing all three Fourier Transform methods:

In an ideal world, we would run the algorithms on a large dataset that contained images of differing sizes. However, in the absence of such dataset, we will choose the following approach for implementation of the analysis -

1. First, assume the size of the image is MxN. Scale the image to a square size, say N. if original image is not a square image. Calculate the run time for the three methods for the square image (NxN). Use bilinear interpolation for the scaling.
2. Now, scale the image by [0.5, 0.5]. Use bilinear interpolation for scaling.
3. Calculate the Fourier transform using all three methods, note the timings for each.
4. If scale == 1 x 1, then stop, else, repeat from step 2.

5. Repeat steps 1-4 **10 times** for one image. Take the average of the 10 runs as the reference runtime for that image. You can cache the scaled versions of the image to speed up the computation of the overall program.
6. Collect the data for at least 5 images from the test dataset.
7. Plot the timings Vs the image size N, to compare the three methods.

The above approach should give you a good statistical dataset to compare the "real-world" performance of the three methods.

Part (E) Extra Credit - Compression using FFT (10 points)

Implement a trivial compression scheme by allowing the user to eliminate some frequency components. For e.g. Implement a thresholding scheme that only allows certain values of (u,v) to pass through. The user may manipulate two sliders (in the GUI) to choose the lower and upper range of the frequencies that are passed through. You should zero-out the frequency values outside this range when computing the inverse Fourier transform. How many frequency components can you eliminate before you start losing the image details and the output is no longer similar to the original? Provide image examples in the README document.
Comment on the distribution of "power" in the image as we move from the origin (u=0 and v=0) towards higher frequency components on the edge of the image.(10 points)

Deliverable : For submitting your assignment, create an archive that is named as follows

CS370_PP3_<*digipen_login_id*>**.zip**

Include the following in the archive:

(a) ./src/ - complete source code of the application including any project files that you create (do NOT include the **.ncb** file if compiling on Windows, but include the solution **.sln** file).

(b) ./doc/README - Refer to Part (D) for details on the report. If you have implemented the extra-credit part (2), comment on the distribution of "power" in the image as we move from the origin (u=0 and v=0) towards higher frequency components on the edge of the image.

Failure to submit the deliverable in the required format, or not provide enough examples in the README file, will result in a zero assignment grade.

Late Submission : Refer to the course outline for the late submission guidelines.

Grade Sheet for Programming Project 3 (CS370/570)

Item	Points Possible	Points Awarded	Comments
Part A			
Direct method	4		
Separable method	6		
Part B			
FFT Implementation	20		
Part C			
Intensity slicing	10		
Part D			
Analysis of the three methods	20		
Part E - Extra Credit			
Extra Credit - Image compression	10		
Penalties, if any			
TOTAL GRADE	**60**		